Architecture

This document describes the system architecture, design patterns, and technical decisions for the TroveTrends Admin Dashboard.

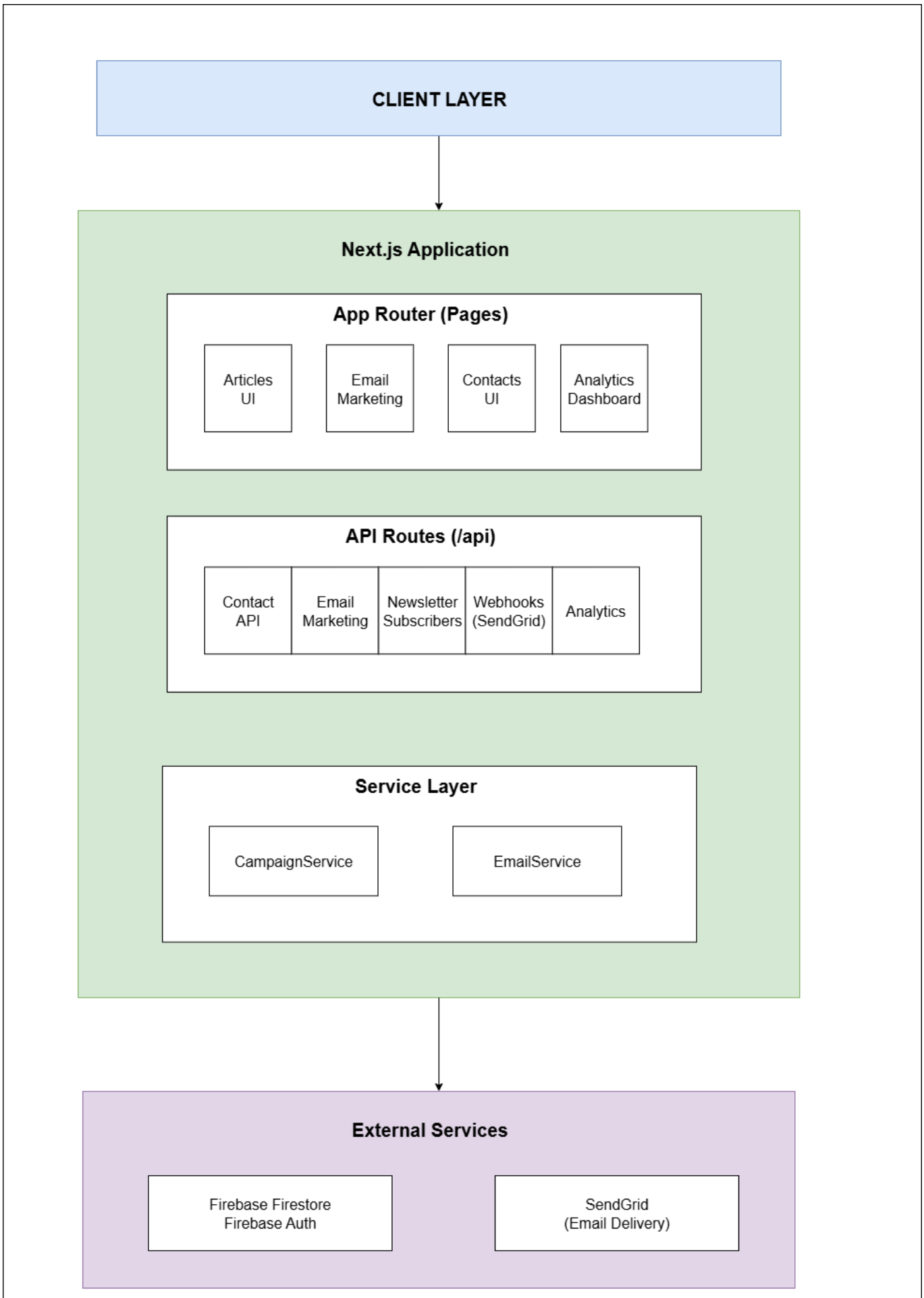
Table of Contents

- System Overview
- Technology Stack
- Application Architecture
- Data Flow
- Component Architecture
- Service Layer
- State Management
- Authentication Architecture
- Analytics Dashboard
- External Integrations
- Design Decisions
- Performance Considerations
- Security Architecture
- Future Considerations

System Overview

TroveTrends Admin is a Next.js 14 application that serves as the administration backend for the TroveTrends platform. It follows a monolithic architecture with clear separation of concerns between UI components, business logic services, and data access layers.

High-Level Architecture


CLIENT LAYER
Next.js Application
App Router (Pages)
Articles UI
Email Marketing
Contacts UI
Analytics Dashboard
API Routes (/api)
Contact API
Email Marketing
Newsletter Subscribers
Webhooks (SendGrid)
Analytics
Service Layer
CampaignService
EmailService
External Services
Firebase Firestore
Firebase Auth
SendGrid
(Email Delivery)

Technology Stack

Core Framework

Technology	Version	Purpose
Next.js	14.2.4	Full-stack React framework
React	18.x	UI component library
TypeScript	5.8.3	Type-safe development
Node.js	18.x+	Runtime environment

UI Layer

Technology	Purpose
Tailwind CSS 3.4.1	Utility-first CSS framework
Lucide React	Icon library
ApexCharts	Data visualization
React Quill	Rich text editing
Flatpickr	Date picker component

Backend Services

Technology	Purpose
Firebase Admin SDK 12.0	Server-side Firebase operations
Firebase Client SDK 11.2	Client-side authentication
SendGrid	Email delivery service
Express (optional)	Custom server for advanced use cases

Data Layer

Technology	Purpose
Firestore	NoSQL document database
Firebase Auth	User authentication

Application Architecture

Directory Structure

```
src/
├── app/                    # Next.js App Router
│   ├── api/                # API route handlers
│   │   ├── analytics/      # Analytics dashboard endpoint
│   │   ├── contact/        # Contact management endpoints
│   │   ├── email-marketing/  # Email marketing endpoints
│   │   ├── newsletter-subscribers/
│   │   └── webhook/        # SendGrid webhooks
│   ├── articles/           # Article management feature
│   ├── auth/               # Authentication pages
│   ├── email-marketing/    # Email marketing feature
│   ├── contacts/           # Contact management feature
│   └── layout.tsx          # Root layout
│
├── components/             # React components
│   ├── Articles/           # Article-specific components
│   ├── Charts/             # Analytics chart components
│   │   ├── ChartOne.tsx    # Campaign performance area chart
│   │   ├── ChartTwo.tsx    # Weekly metrics bar chart
│   │   └── ChartThree.tsx  # Engagement donut chart
│   ├── Dashboard/          # Dashboard components
│   │   └── E-commerce.tsx  # Main analytics dashboard
│   ├── EmailMarketing/     # Email marketing components
│   ├── Header/             # Navigation header
│   ├── Layouts/            # Layout components
│   ├── Maps/               # Map and stats components
│   ├── Sidebar/            # Navigation sidebar
│   ├── Tables/             # Data table components
│   └── common/             # Shared components
│
├── services/               # Business logic layer
│   └── EmailMarketing/     # Email marketing services
│
├── types/                  # TypeScript type definitions
│   ├── Analytics.ts        # Analytics dashboard types
```

```
│   ├── Article.ts            # Article types
│   ├── EmailMarketing/       # Email marketing types
│   └── ...                   # Other domain types
│
├── utils/                    # Utility functions
│   ├── firebase-admin.ts     # Server-side Firebase
│   ├── firebaseClient.js     # Client-side Firebase
│   └── emailService.ts       # Email utilities
│
└── hooks/                    # Custom React hooks
```

Layer Responsibilities

Presentation Layer (components/)
- React components for UI rendering
- User interaction handling
- Local component state management
- Form validation and display logic

API Layer (app/api/)
- HTTP request/response handling
- Request validation
- Response formatting
- CORS configuration
- Error handling

Service Layer (services/)
- Business logic implementation
- Data transformation
- Complex operations coordination
- Transaction management

Data Access Layer (utils/)
- Firebase initialization and configuration
- Database query abstractions
- External service clients

Data Flow

Read Operation Flow

```
User Action
    │
```

```
    ▼
React Component
    │
    ├── Client-side: Firebase SDK direct query
    │   │
    │   └── Firestore ──► Data ──► Component State ──► UI Update
    │
    └── Server-side: API route call
        │
        ▼
    API Route Handler
        │
        ▼
    Service Layer (if complex logic)
        │
        ▼
    Firebase Admin SDK
        │
        ▼
    Firestore
        │
        ▼
    JSON Response ──► Component State ──► UI Update
```

Write Operation Flow

```
User Submits Form
    │
    ▼
React Component (validation)
    │
    ▼
API Route (POST/PUT/DELETE)
    │
    ▼
Service Layer
    │
    ├── Data validation
    ├── Business rule enforcement
    └── Transaction coordination
        │
        ▼
Firebase Admin SDK
    │
    ▼
Firestore (write)
    │
    ▼
Response ──► UI Update / Redirect
```

Email Campaign Flow

```
Campaign Created
    │
    ▼
CampaignService.create()
    │
    ▼
Firestore (save campaign)
    │
    ▼
User triggers send
    │
    ▼
CampaignService.send()
    │
    ├── Fetch subscriber list
    ├── Apply segment filters
    ├── Generate personalized emails
    │
    ▼
EmailService.sendBulk()
    │
    ▼
SendGrid API
    │
    ▼
SendGrid Webhook ──► Update campaign stats
```

Analytics Dashboard Flow

```
Dashboard Page Load
    │
    ▼
E-commerce Component (useEffect)
    │
    ▼
GET /api/analytics?period=month
    │
    ▼
API Route Handler
    │
    ▼
Promise.all() - Parallel Queries
    │
    ├── getKPISummary()
    │   ├── newsletter_subscribers.count()
    │   ├── emailCampaigns.where('completed')
    │   ├── articles.get() (viewCount, affiliateClicks)
```

```
    │   └── contacts.count()
    │
    ├── getEngagementBreakdown()
    │   └── newsletter_subscribers.where('active')
    │       └── Group by engagementScore thresholds
    │
    ├── getSubscriberStatusBreakdown()
    │   └── Parallel count queries by status
    │
    ├── getArticlePerformance()
    │   └── articles.orderBy('viewCount').limit(10)
    │
    ├── getWeeklyMetrics()
    │   └── Aggregate last 7 days by day-of-week
    │
    ├── getMonthlyMetrics()
    │   └── Aggregate last 12 months
    │
    ├── getCampaignPerformanceTimeSeries()
    │   └── Monthly opens, clicks, sent, delivered
    │
    └── getSubscriberGrowthTimeSeries()
        └── Monthly new subscribers vs unsubscribes
    │
    ▼
AnalyticsDashboardData Response
    │
    ▼
Component State Update
    │
    ▼
ApexCharts Render
    ├── ChartOne (Area) - Campaign Performance
    ├── ChartTwo (Bar) - Weekly Metrics
    ├── ChartThree (Donut) - Engagement Breakdown
    ├── TableOne - Top Articles
    └── MapOne - Subscriber Status
```

Component Architecture

Layout System

```
RootLayout (app/layout.tsx)
    │
    └── DefaultLayout (components/Layouts/)
        │
        ├── Sidebar (components/Sidebar/)
        │   ├── SidebarItem
        │   └── SidebarDropdown
```

```
        │
        ├── Header (components/Header/)
        │   ├── DropdownUser
        │   ├── DropdownNotification
        │   └── DarkModeSwitcher
        │
        └── Main Content Area
            └── Page Components
```

Dashboard Component Structure

```
E-commerce Dashboard (components/Dashboard/E-commerce.tsx)
    │
    ├── KPI Cards Row
    │   ├── Total Subscribers (with growth rate)
    │   ├── Campaigns Sent
    │   ├── Average Open Rate
    │   └── Total Article Views
    │
    ├── Charts Row
    │   ├── ChartOne (components/Charts/ChartOne.tsx)
    │   │   └── Area chart: Campaign opens vs clicks (12 months)
    │   │
    │   └── ChartTwo (components/Charts/ChartTwo.tsx)
    │       └── Bar chart: Weekly metrics breakdown
    │
    ├── Secondary Row
    │   ├── ChartThree (components/Charts/ChartThree.tsx)
    │   │   └── Donut chart: Engagement distribution
    │   │
    │   ├── MapOne (components/Maps/MapOne.tsx)
    │   │   └── Subscriber status breakdown stats
    │   │
    │   └── TableOne (components/Tables/TableOne.tsx)
    │       └── Top performing articles table
    │
    └── ChatCard (components/Chat/ChatCard.tsx)
        └── Recent activity / notifications
```

Component Patterns

Container/Presentation Pattern

Page components act as containers that fetch data and manage state, while child components focus on presentation:

```
// Container (page.tsx)
export default function ArticlesPage() {
```

```
  return (
    <DefaultLayout>
      <ArticlesDashboard />  {/* Presentation component */}
    </DefaultLayout>
  );
}

// Presentation (ArticlesDashboard.tsx)
export default function ArticlesDashboard() {
  const [articles, setArticles] = useState<Article[]>([]);
  // Component handles its own data fetching and state
}
```

Compound Components

Complex UI elements use compound component patterns for flexibility:

```
// Sidebar with dropdown items
<Sidebar>
  <SidebarItem label="Dashboard" />
  <SidebarDropdown label="Email Marketing">
    <SidebarItem label="Campaigns" />
    <SidebarItem label="Templates" />
  </SidebarDropdown>
</Sidebar>
```

Service Layer

Service Pattern

Services are implemented as classes with static methods for stateless operations:

```
// services/EmailMarketing/CampaignService.ts
export class CampaignService {
  static async create(data: CreateCampaignInput): Promise<Campaign> {
    // Validation
    // Business logic
    // Database operation
    // Return result
  }

  static async send(campaignId: string): Promise<void> {
    // Fetch campaign
    // Get subscribers
    // Send emails
    // Update statistics
```

```
  }
}
```

Service Responsibilities

CampaignService
- Campaign CRUD operations
- Campaign scheduling logic
- Subscriber filtering
- Statistics aggregation
- Batch processing coordination

EmailService
- Email template rendering
- Variable substitution
- SendGrid API communication
- Delivery tracking

State Management

Current Approach

The application uses a combination of:

1. **React Local State**: `useState` for component-level state
2. **Custom Hooks**: `useColorMode`, `useLocalStorage` for cross-cutting concerns
3. **Server State**: Firestore as the source of truth

State Categories

State Type	Storage	Example
UI State	useState	Modal open/closed, form inputs
Theme	localStorage + useState	Dark/light mode
Server Data	Firestore	Articles, campaigns, subscribers
Session	Firebase Auth	User authentication state

Data Fetching Pattern

```
// Component-level data fetching
function ArticlesDashboard() {
  const [articles, setArticles] = useState<Article[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchArticles = async () => {
      const response = await fetch('/api/articles');
      const data = await response.json();
      setArticles(data);
      setLoading(false);
    };
    fetchArticles();
  }, []);
}
```

Authentication Architecture

Authentication Flow

```
User Login Request
    |
    ▼
Firebase Auth (Client SDK)
    |
    ├── Email/Password
    └── Google OAuth
        |
        ▼
Firebase ID Token Generated
    |
    ▼
Token stored in browser
    |
    ▼
API Requests include Authorization header
    |
    ▼
API Route verifies token (Firebase Admin SDK)
    |
    ▼
Custom claims checked for RBAC
    |
```

```
    ▼
Request processed or rejected
```

Role-Based Access Control

```
// Firebase custom claims structure
interface UserClaims {
  admin?: boolean;
  editor?: boolean;
  viewer?: boolean;
}

// Middleware verification
async function verifyAdmin(token: string): Promise<boolean> {
  const decodedToken = await admin.auth().verifyIdToken(token);
  return decodedToken.admin === true;
}
```

Analytics Dashboard

Overview

The analytics dashboard provides real-time insights into subscriber engagement, campaign performance, and article metrics. All data is aggregated from Firestore collections using parallel queries for optimal performance.

API Endpoint

```
GET /api/analytics?period=month
```

Returns `AnalyticsDashboardData` with all metrics aggregated from multiple Firestore collections.

Type Definitions

```
// src/types/Analytics.ts

interface AnalyticsDashboardData {
  kpiSummary: KPISummary;
  campaignPerformance: CampaignPerformanceData;
  subscriberGrowth: SubscriberGrowthData;
  engagementBreakdown: EngagementBreakdown;
  subscriberStatusBreakdown: SubscriberStatusBreakdown;
```

```
  articlePerformance: ArticlePerformanceData;
  weeklyMetrics: PeriodMetrics[];
  monthlyMetrics: PeriodMetrics[];
  lastUpdated: string;
}

interface KPISummary {
  totalSubscribers: number;
  subscriberGrowthRate: number;      // % change from last period
  totalCampaignsSent: number;
  averageOpenRate: number;
  averageClickRate: number;
  totalArticleViews: number;
  totalAffiliateClicks: number;
  totalContacts: number;
}

interface EngagementBreakdown {
  highEngagement: number;    // engagementScore > 70
  mediumEngagement: number;  // 30-70
  lowEngagement: number;     // < 30
  inactive: number;          // no activity
}
```

Metrics Computation

Metric	Source Collection	Computation Method
Total Subscribers	newsletter_subscribers	`count()` query
Subscriber Growth Rate	newsletter_subscribers	Compare 30-day vs previous 30-day counts
Campaigns Sent	emailCampaigns	Count where status = 'completed'
Average Open Rate	emailCampaigns	Sum(opens) / Sum(sent) * 100
Average Click Rate	emailCampaigns	Sum(clicks) / Sum(sent) * 100
Article Views	articles	Sum of all viewCount fields
Affiliate Clicks	articles	Sum of all affiliateClicks fields
Engagement Breakdown	newsletter_subscribers	Group by engagement.engagementScore thresholds

Chart Components

Component	Chart Type	Data Source	Metrics Displayed
ChartOne	Area Chart	campaignPerformance	Opens & clicks over 12 months
ChartTwo	Bar Chart	weeklyMetrics	Daily opens, clicks, new subscribers
ChartThree	Donut Chart	engagementBreakdown	High/Medium/Low/Inactive segments
TableOne	Data Table	articlePerformance.topArticles	Title, views, affiliate clicks
MapOne	Stats Cards	subscriberStatusBreakdown	Active/Pending/Unsubscribed/ Bounced

Performance Optimizations

1. **Parallel Queries**: All data fetches use `Promise.all()` for concurrent execution
2. **Count Queries**: Uses Firestore `count()` aggregation instead of fetching documents
3. **Limited Results**: Top articles limited to 10, categories to 5
4. **Computed on Server**: All aggregations performed server-side to minimize client processing

Engagement Score Thresholds

Level	Score Range	Description
High	> 70	Regularly opens and clicks
Medium	30-70	Occasional engagement
Low	1-29	Rarely engages
Inactive	0	No recorded activity

External Integrations

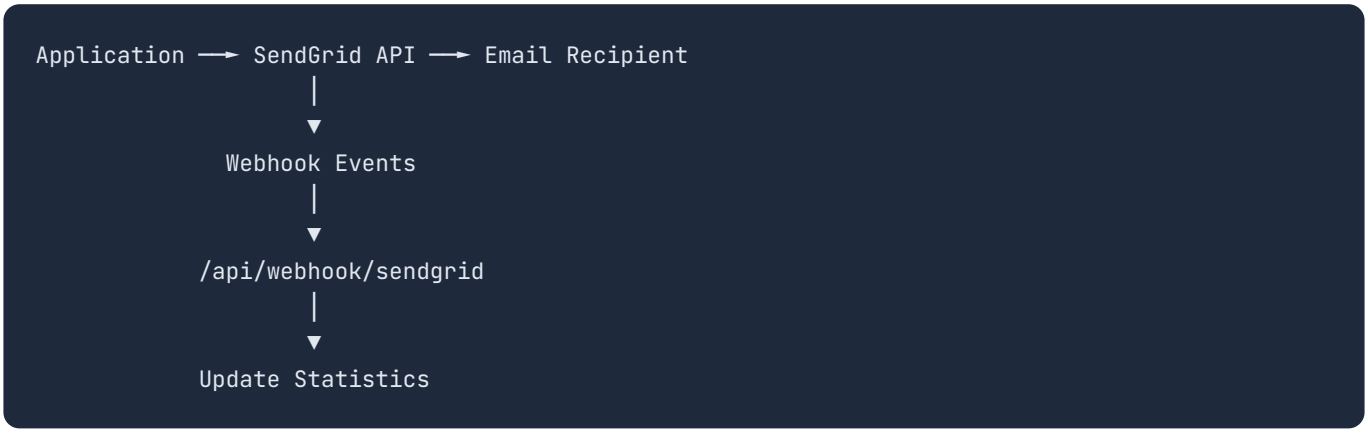
Firebase Integration

Client-Side (firebaseClient.js)
- Authentication state management
- Real-time data subscriptions (if used)
- Client-side queries for public data

Server-Side (firebase-admin.ts)
- Secure database operations
- Token verification
- Admin operations (user management)
- Background jobs

SendGrid Integration

Email Delivery



Tracked Events
- Delivered
- Opened
- Clicked
- Bounced
- Unsubscribed
- Complained

Frontend Integration

The admin dashboard shares data with TroveTrends Frontend through:

1. **Shared Firestore Database**: Both applications read/write to same collections

2. **API Endpoints**: Frontend calls admin API for newsletter subscriptions
3. **Consistent Data Schema**: Shared type definitions ensure compatibility

Design Decisions

Why Next.js App Router

- Server-side rendering for better SEO (admin pages)
- API routes eliminate need for separate backend
- File-based routing simplifies navigation structure
- Built-in optimization for production builds

Why Firebase/Firestore

- Real-time capabilities for live updates
- Integrated authentication solution
- Scalable NoSQL document model
- Generous free tier for development
- Simple security rules

Why SendGrid

- High deliverability rates
- Comprehensive webhook system
- Template management
- Analytics and tracking
- Reliable API

Why Tailwind CSS

- Rapid UI development
- Consistent design system
- Dark mode support built-in
- Small production bundle with purging
- Responsive utilities

Why TypeScript

- Catch errors at compile time
- Better IDE support and autocomplete
- Self-documenting code through types
- Easier refactoring
- Improved team collaboration

Performance Considerations

Client-Side

- Code splitting via Next.js dynamic imports
- Image optimization with next/image
- Lazy loading for heavy components (charts, editors)
- ApexCharts rendered client-side only (SSR disabled)

Server-Side

- Firestore query optimization with indexes
- Pagination for large data sets
- Caching strategies for static data
- Analytics API uses parallel queries via `Promise.all()`

Database

- Composite indexes for complex queries
- Denormalization for read-heavy operations
- Batch writes for bulk operations
- Firestore `count()` aggregation for analytics totals

Analytics-Specific Optimizations

Optimization	Implementation
Parallel Fetching	8 concurrent Firestore queries
Aggregation Queries	`count()` instead of document fetch

Optimization	Implementation
Result Limiting	Top 10 articles, 5 categories max
Server-side Computation	All calculations in API route
Time-bounded Queries	12-month and 7-day windows

Security Architecture

Defense Layers

1. **Authentication**: Firebase Auth verifies user identity
2. **Authorization**: Custom claims and Firestore rules control access
3. **Input Validation**: Server-side validation for all inputs
4. **CORS**: Configured allowed origins
5. **Environment Variables**: Secrets stored securely

Security Rules

```
// firestore.rules
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Production rules should restrict access
    match /{document=**} {
      allow read, write: if request.auth ≠ null;
    }
  }
}
```

Future Considerations

Potential Improvements

1. **State Management**: Consider React Query or SWR for server state caching
2. **Testing**: Add unit and integration test coverage
3. **Monitoring**: Implement error tracking and performance monitoring
4. **CI/CD**: Automated testing and deployment pipelines

5. **Caching**: Redis or similar for frequently accessed data

Scalability Path

1. **Database**: Firestore scales automatically
2. **Compute**: Vercel handles auto-scaling
3. **Email**: SendGrid handles high volume
4. **Assets**: CDN for static assets